NURUDEEN ADENIYI

West Hempstead, NY | 646-801-7833 | nadeniyi@outlook.com
linkedin.com/in/nurudeen-adeniyi-06249b9 | github.com/nuru-adeniyi

Senior Software Engineer — Backend, Distributed Systems & System Design

SUMMARY

Senior software engineer who designs and operates the distributed backends businesses depend on — and makes the deliberate architecture decisions behind them. On a small platform team I led the design of GoodLeap's catalog and communications services on AWS: an event-driven messaging platform at ~2.4M emails and ~300K SMS/month, and a loan-pricing engine serving 1,000+ installers. I design for failure and change — transport-agnostic command handlers with built-in fallback paths, cache-aside read models with event-driven invalidation, date-effective data models, multi-tenant isolation, and SLO-based observability — then own them through production debugging and incident response. I've also studied the fundamentals of AI engineering — LLMs, RAG, and agentic tools — through tutorials and a live personal project, and I'm currently expanding my AWS cloud-engineering skills.

SELECTED ARCHITECTURE & SYSTEM DESIGN

- **Transport-agnostic command handlers with a built-in failover path**: designed the communications services so the Kafka consumer and the REST API invoke the same command handler — decoupling business logic from transport. When Kafka went down in production, we kept processing by calling the synchronous API directly against the same handler, with no data loss while the broker was restored.
- **Cache-aside pricing read model with event-driven invalidation**: architected the loan-pricing path to cache an expensive per-org base offer set in Redis, then apply per-request filters (org eligibility, category, date-effective tier) and pricing math (dealer-fee, risk-based/FICO adjustments) on top. Org-tier change events on Kafka invalidate the cache for consistency, and a Polly circuit breaker with timeouts and a database fallback keep pricing responding if Redis is unavailable.
- **Event-driven, serverless messaging platform on AWS**: designed a pipeline of Kafka → SES / Twilio → per-stage SQS queues (transmit, delivery-receipt, inbound) with dead-letter queues at each stage and an encryption boundary for message bodies — sustaining ~2.4M emails and ~300K SMS/month at peak.
- **Date-effective (point-in-time) pricing model with org-tier precedence**: modeled fee-adjustment schedules that return the correct base fee for any given date, configured across DynamoDB and MongoDB, with organization config overriding platform defaults — so pricing stays correct across rate changes and effective-date boundaries.
- **From-scratch, multi-tenant communications data model**: designed the recipient-centric messaging data model (EF Core Fluent API, soft-delete, comm threads) and tenant-scoped SMS event aggregation for isolation across organizations.
- **SLO-style observability**: stood up Datadog monitors that alert on sustained 500-error rates (> 5 minutes) and latency, structured Serilog logging, and end-to-end Kafka latency instrumentation — so regressions surface before customers feel them.

TECHNICAL SKILLS

Languages: C# / .NET (incl. .NET 8), SQL (T-SQL), JavaScript / TypeScript, Node.js; Python, PowerShell

Architecture & Design: Distributed systems, microservices, event-driven architecture, CQRS / MediatR, Clean Architecture & domain-driven design, cache-aside, circuit breaker & graceful degradation, dead-letter queues, idempotent & transport-agnostic handlers, multi-tenancy, date-effective/temporal modeling

Backend & Data: ASP.NET Core (MVC & Web REST API), Entity Framework Core, FluentValidation, Polly, Kafka (AWS MSK), Redis, REST & webhook APIs, SQL Server, MongoDB, DynamoDB, Twilio

Cloud & DevOps: AWS (EC2, ECS Fargate, Lambda, S3, RDS, SES, SNS, SQS, VPC, ELB, CloudWatch, IAM), Docker, Terraform, GitHub Actions, Git, trunk-based delivery; Agile / Scrum, Jira, Confluence

Front End: React, Next.js, Angular, HTML5, CSS, Bootstrap

Observability & Testing: Datadog (SLO alerting), Serilog structured logging, CloudWatch, ECS health checks; xUnit, Moq, unit & integration test automation

Leadership & Collaboration: Mentoring and code review; clear technical communication — authored the API docs, design specs, and demos other teams built against; cross-functional partnership with product and platform teams; ownership of outcomes from design through delivery

AI & LLM: Uses Claude Code & GitHub Copilot daily; integrated OpenAI GPT (news-sentiment classification) into a live personal project; studied the fundamentals — RAG, vector databases, LangChain / LangGraph, agents / MCP — through tutorials

PROFESSIONAL EXPERIENCE

Senior Software Engineer Apr 2022 – Jul 2026

GoodLeap | *Roseville, CA (Remote)*

Part of a small platform team (2–3 engineers) that designed, built, and maintained GoodLeap's catalog and communications services and unblocked other product squads. I took the lead on much of the catalog and comms work — service design and data modeling, code review, testing, and the deepest root-cause investigations. (Architecture highlights above.)

Reliability, performance & incident response

- Traced a 7-minute delay between a Kafka message arriving and being processed to a changed stored procedure holding a database lock — after ruling out consumer lag, thread starvation, and GC pauses — and brought it from ~7 minutes to instant.
- Found an API sitting at 20+ seconds in Datadog, traced it to an unnecessary loop, and brought the endpoint from ~20 seconds to milliseconds; tuned slow SQL Server queries via execution-plan analysis and indexing.
- Traced 20,000+ failed emails in CloudWatch to AWS's ~14/second send-rate limit during high-volume sends; worked with AWS to raise it, then queried and requeued every failed message so none were lost.
- Fixed a concurrency race where a shared EF Core DbContext was reused across parallel queries, causing intermittent 500s — isolating context usage to eliminate the errors under load.

Cost, correctness & data integrity

- Right-sized the ECS fleet from 4 instances to 2 after seeing steady-state CPU stay under ~30% — no throughput loss — and removed redundant status updates to cut PUT volume 50–60% and stabilize containers under peak load.
- Caught an AWS RDS SQL Server instance nearing capacity — a close call that could have stopped email/SMS delivery — fixed it and added an alert, preventing a platform-wide outage.
- Found and fixed a floating-point rounding bug in loan-fee calculations that was quietly underbilling borrowers on every affected fee; added FICO min/max, duplicate-rate-spread, and expiry validation guards that eliminated whole classes of silent data-integrity bugs.

Platform, compliance & team enablement

- Built core platform (shared-services) pieces of the catalog's Risk-Based Pricing (RBP) add-on — a cross-team initiative serving 1,000+ installers — so each applicant gets the offer priced for their FICO score, including a sub-650 counter-offer API and a dealer-fee override integrated with pricing.
- Removed a PII compliance risk (email addresses, subject lines, and bodies leaking into logs), built the SMS opt-in/opt-out (STOP) subscription system establishing the carrier-compliance boundary, and hardened exception propagation so failures surface fully instead of being silently swallowed.
- Wrote and maintained Confluence documentation and design specs (API references, sample request/response contracts) for every service and ran demos so other teams could integrate without back-and-forth; replaced reflection-based mapping (AutoMapper) with compile-time Mapperly, and automated a weekly deploy-commit digest for QA in our trunk-based workflow.

.NET Software Developer Jun 2015 – Mar 2022

Blackbaud | *Charleston, SC (Remote)*

- Introduced microservice REST Web APIs built on SOLID principles, making code easier to unit-test and giving front-end developers cleaner, more reliable services to build on.
- Invented a batch-upload process for school billing that replaced slow manual entry — recognized company-wide with a “We Invent Possibilities” award; designed and built a customer-facing full-stack web application that grew product revenue.
- Helped stand up the DevOps pipeline (daily builds, releases, code review) on Azure, and mentored and coordinated offshore developers.

.NET Software Developer (Full-Stack) Nov 2008 – Jun 2015

Smart LLC | *New York, NY*

- Built and maintained a full-stack ASP.NET / C# platform on SQL Server for financial-aid management; automated monthly invoicing with a Windows service and served as a hands-on team lead mentoring newer developers.

SIDE PROJECT

Automated Algorithmic Trading Platform — Python, AWS, LLM github.com/nuru-adeniyi/alpaca-trade

- Built and deployed a Python (Flask) webhook service on AWS ECS Fargate (Docker, Terraform, S3 state, CloudWatch auto-scaling, ~18 scheduled GitHub Actions workflows) turning TradingView signals into live brokerage orders; designed a broker-agnostic engine (five brokerages) with backoff retries, rate limiting, and a watchdog that restarts stalled workers within 30s.
- Integrated an LLM (OpenAI GPT) that classifies live news sentiment and triggers trades — hands-on experience wiring a large-language model into an event-driven production pipeline.

EDUCATION & CERTIFICATIONS

B.S., Computer Science — York College (CUNY), Queens, NY

AWS Certified Solutions Architect – Associate (SAA) | Microsoft Certified Application Developer (MCAD) | Currently expanding AWS cloud-engineering skills